

Лабораторна робота №3

REST API з Express.js та інтеграційне тестування

Мета роботи: набуття практичних навичок розробки RESTful API на Express.js з TypeScript та написання інтеграційних тестів з використанням Supertest

3.1. Завдання 1: Ініціалізація проєкту

Створіть новий Node.js-проєкт з усіма необхідними залежностями.

Вимоги:

- Ініціалізуйте проєкт (`npm init -y`) та встановіть залежності:
 - runtime: `express`, `cors`, `zod`
 - dev: `typescript`, `@types/*` для кожного пакету, `ts-node`, `jest`, `ts-jest`, `@types/jest`, `supertest`, `@types/supertest`
- Налаштуйте `tsconfig.json` та `jest.config.js`.
- Додайте до `package.json` скрипти: `dev`, `build`, `test`, `test:coverage`.
- Структура проєкту:

```
project/
├─ src/
│  ├─ schemas/entity.schema.ts
│  ├─ storage/entity.ts
│  ├─ routes/entity.ts
│  ├─ middleware/
│  ├─ app.ts
│  └─ server.ts
├─ tests/entity.test.ts
├─ tsconfig.json
├─ jest.config.js
└─ package.json
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.21.121.23.000 – Лр.3		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Сорока А.О.			Звіт з лабораторної роботи №3	Літ.	Арк.
Перевір.		Фуріхата Д. В.					1
Реценз.						ФІКТ, гр. ВТ-23-1	
Н. Контр.							
Зав.каф.							

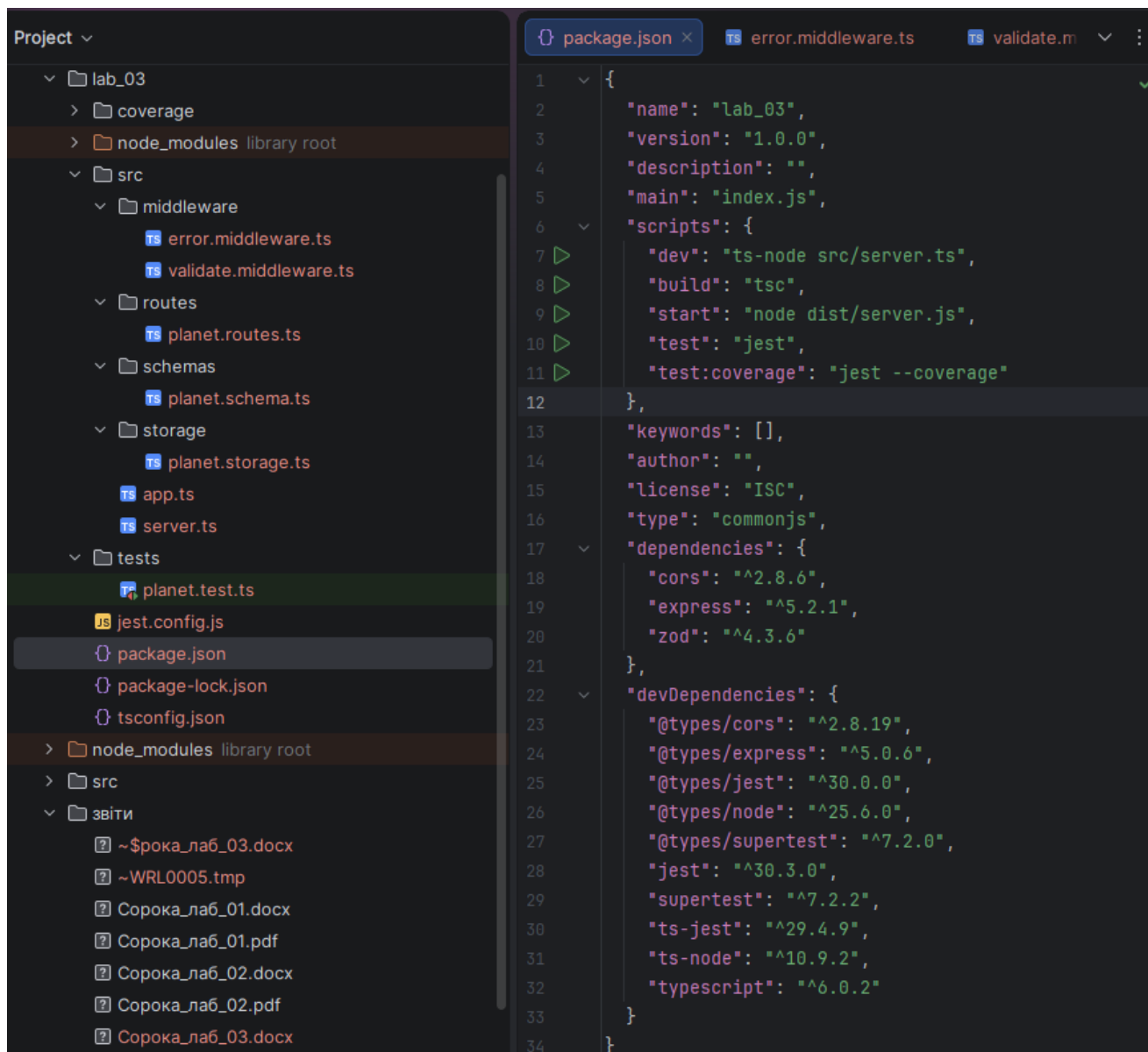


Рис. 1. Структура проекту

3.2. Завдання 2: Схеми та типи сутності

Створіть схему для роботи з обраною сутністю. Оберіть сутність із домену, який вам знайомий — книга, рецепт, фільм, гравець, планета тощо. Чим краще ви розумієте предметну область, тим легше вийде зробити схему. Не треба повторювати приклад з лекції :)

Вимоги:

- Схема створення (createSchema) повинна містити:

- обов'язкове текстове поле з назвою, що відповідає вашій сутності (title для книги, name для персонажа, designation для планети тощо) — 1–100 символів;
- опціональне поле опису — до 500 символів;
- мінімум 2 додаткових поля, що відображають реальні атрибути вашої сутності. Поле має відповідати на питання: «що корисно знати про цей об'єкт?» Наприклад, для книги це може бути рік видання (число зі зрозумілими обмеженнями) та жанр (список заготовлених значень); для рецепту — час приготування в хвилинах та рівень складності. Поля типу isActive чи count без конкретного змісту не зараховуються.
- Схема оновлення (updateSchema) — всі поля опціональні.
- Тип збереженої сутності (Entity) — виводиться з createSchema (z.infer) і розширюється серверними полями: id, createdAt, updatedAt.

Файл: src/schemas/planet.schema.ts

```
import {z} from "zod";

const PLANET_TYPES = ['Terrestrial', 'Gas Giant', 'Ice Giant', 'Dwarf Planet'] as const;

// Post запит
export const createPlanetSchema = z.object({
  name: z.string().min(1, "Назва планети не може бути порожньою").max(100, 'Назва занадто довга'),
  description: z.string().max(500, "Опис занадто довгий").optional(),

  type: z.enum(PLANET_TYPES, {
    message: "Допустимі типи: Terrestrial, Gas Giant, Ice Giant, Dwarf Planet"
  }),
  // Маса відносно Землі (Земля = 1).
  massEarth: z.number().positive("Маса має бути більшою за 0"),
});
// Patch запит
export const updatePlanetSchema = createPlanetSchema.partial();

export type CreatePlanetInput = z.infer<typeof createPlanetSchema>;
export type UpdatePlanetInput = z.infer<typeof updatePlanetSchema>;

export type PlanetEntity = CreatePlanetInput & {
  id: string,
  createdAt: Date,
  updatedAt: Date
}
```

3.3. Завдання 3: Модуль зберігання (in-memory storage)

Реалізуйте CRUD-модуль на основі Map.

Вимоги:

- Реалізуйте модуль на основі Map<string, Entity>. Назви функцій та їх сигнатури — на ваш розсуд.
- Модуль повинен підтримувати: отримання всіх записів, отримання за id, створення, оновлення (з оновленням updatedAt), видалення.
- Передбачте можливість скидання стану сховища — вона знадобиться для ізоляції тестів.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.3	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

Файл: src/storage/planet.storage.ts

```
import crypto from "crypto";
import {PlanetEntity, CreatePlanetInput, UpdatePlanetInput} from "../schemas/planet.schema";
import {type} from "node:os";
const now = new Date();
export interface PlanetFilters {
  type?: string,
  minMassEarth?: number,
}

let storage = new Map<string, PlanetEntity>();

export const PlanetStorage = {
  getAll(filters?: PlanetFilters): PlanetEntity[] {
    let planets = Array.from(storage.values());
    if (filters?.type) {
      planets = planets.filter(p => p.type === filters.type);
    }
    if (filters?.minMassEarth !== undefined) {
      planets = planets.filter(p => p.massEarth >= Number(filters.minMassEarth));
    }
    return planets;
  },
  getById(id: string): PlanetEntity | undefined {
    return storage.get(id);
  },
  create( data: CreatePlanetInput): PlanetEntity {
    const id = crypto.randomUUID()
    const now = new Date();

    const newPlanet = {
      ...data,
      id: id,
      createdAt: now,
      updatedAt: now,
    }
    storage.set(id, newPlanet);
    return newPlanet;
  },
  update(id: string, data: UpdatePlanetInput): PlanetEntity | null {
    const existing = storage.get(id);
    if (!existing){
      return null;
    }
    const updatedPlanet = {
      ...existing,
      ...data,
      updatedAt: new Date(),
    }
    storage.set(id, updatedPlanet);
    return updatedPlanet;
  },
  delete(id: string): boolean {
    return storage.delete(id);
  },
  reset(): void {
    storage.clear();
  }
}
```

3.4. Завдання 4: CRUD-маршрути та middleware

Реалізуйте Express Router з повним набором CRUD-маршрутів, validate middleware та errorHandler.

Вимоги:

- Реалізуйте validate(schema) middleware — парсить тіло запиту через схему і викидає помилку, якщо дані невалідні.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.3	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		4

- Реалізуйте errorHandler — розрізняйте помилки валідації (ZodError, статус 400) та інші помилки (статус 500). Формат відповіді при помилці — на ваш розсуд, але він має бути інформативним.
- Підтримайте операції: отримання всіх записів, отримання за id, створення, оновлення, видалення.
- Обирайте статус-коди відповідно до HTTP-семантики: 200, 201, 204, 400, 404 тощо.
- При створенні генеруйте id через crypto.randomUUID(), виставляйте дату createdAt/updatedAt.

Файл: src/middleware/validate.middleware.ts

```
import { Request, Response, NextFunction } from 'express';
import { z } from 'zod';

export const validate = (schema: z.ZodTypeAny) => {
  return async (req: Request, res: Response, next: NextFunction) => {
    try {
      req.body = await schema.parseAsync(req.body);
      next();
    } catch (error) {
      next(error);
    }
  }
};
```

Файл: src/middleware/error.middleware.ts

```
import { Request, Response, NextFunction } from 'express';
import { ZodError } from "zod";

export const errorHandler = (err: any, req: Request, res: Response, next: NextFunction) => {
  if (err instanceof ZodError) {
    return res.status(400).json({
      status: 'error',
      message: 'Помилка валідації даних',
      errors: err.issues
    });
  }
  console.log('Unhandled error');
  return res.status(500).json({
    status: 'error',
    message: 'Внутрішня помилка сервера'
  })
}
```

Файл: src/routes/planet.routes.ts

```
import { Router, Request, Response } from "express";
import { PlanetStorage } from "../storage/planet.storage";
import { validate } from "../middleware/validate.middleware";
import { createPlanetSchema, updatePlanetSchema } from "../schemas/planet.schema";

const router = Router();

// Завдання 3.7 специфічний маршрут важкі планети
router.get('/heavy', (req: Request, res: Response) => {
  const heavyPlanets = PlanetStorage.getAll({minMassEarth: 10});
  res.status(200).json(heavyPlanets);
})
// всі планети
router.get('/', (req: Request, res: Response) => {
  const {type,minMassEarth} = req.query;

  const planets = PlanetStorage.getAll({
    type: type as string,
    minMassEarth: minMassEarth ? Number(minMassEarth) : undefined,
  });
});
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.3	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    res.status(200).json(planets);
  })
  // по айди
  router.get('/:id', (req: Request<{id: string}>, res: Response) => {
    const planet = PlanetStorage.getById(req.params.id);

    if (!planet) {
      res.status(404).json({
        message: 'Планету не знайдено',
      })
    }

    res.status(200).json(planet);
  })
  // створити нову планету
  router.post('/', validate(createPlanetSchema), (req: Request, res: Response) => {
    const newPlanet = PlanetStorage.create(req.body);
    res.status(201).json(newPlanet);
  });
  //оновити планету
  router.patch('/:id', validate(updatePlanetSchema), (req: Request<{id: string}>, res: Response) => {
    const updatedPlanet = PlanetStorage.update(req.params.id, req.body);

    if (!updatedPlanet) {
      return res.status(404).json({ message: 'Планету не знайдено' });
    }

    res.status(200).json(updatedPlanet);
  });
  // видалити
  router.delete('/:id', (req: Request<{id: string}>, res: Response) => {
    const isDeleted = PlanetStorage.delete(req.params.id);

    if (!isDeleted) {
      return res.status(404).json({ message: 'Планету не знайдено' });
    }

    res.status(204).send();
  });
}

export default router;

```

3.5. Завдання 5: Розділення app.ts та server.ts

Розділіть конфігурацію Express та запуск сервера.

Вимоги:

- app.ts — створює і налаштовує Express, **не викликає** listen(), експортує app.
- server.ts — імпортує app, викликає listen().
- Дотримуйтесь правильного порядку підключення middleware.

Файл: src/app.ts

```

import express from 'express';
import cors from 'cors';
import planetRoutes from './routes/planet.routes';
import { errorHandler } from './middleware/error.middleware';

const app = express();

app.use(cors());
app.use(express.json());

app.use('/api/planets', planetRoutes);
app.use(errorHandler);

export default app;

```

Файл: src/server.ts

```

import app from './app';

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server started on port ${PORT}`);
  console.log(`http://localhost:${PORT}/api/planets`);
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.3	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

3.6. Завдання 6: Інтеграційні тести з Supertest

Напишіть інтеграційні тести для вашого API.

Вимоги:

- Використовуйте `beforeEach` для очистки пам'яті та ізоляції тестів.
- Покрийте тестами всі маршрути: успішні сценарії та граничні випадки (не знайдено, невалідні дані тощо).
- Перевіряйте статус-коди, структуру відповідей і заголовки там, де це доречно.
- Загальна кількість тестів — не менше 12.

Файл: `tests/planet.test.ts`

```
import request from 'supertest';
import app from '../src/app';
import { PlanetStorage } from '../src/storage/planet.storage';

const validPlanet = {
  name: 'Earth',
  description: 'Our home',
  type: 'Terrestrial',
  massEarth: 1
};

const heavyPlanet = {
  name: 'Jupiter',
  type: 'Gas Giant',
  massEarth: 317.8
};

describe('Planet REST API Integration Tests', () => {
  // ОЧИЩЕННЯ БАЗИ ПЕРЕД КОЖНИМ ТЕСТОМ (Вимога Завдання 6)
  beforeEach(() => {
    PlanetStorage.reset();
  });

  describe('POST /api/planets', () => {
    it('повинен створити нову планету і повернути статус 201', async () => {
      const res = await request(app)
        .post('/api/planets')
        .send(validPlanet)
        .expect(201);

      expect(res.body).toHaveProperty('id');
      expect(res.body.name).toBe(validPlanet.name);
      expect(res.body).toHaveProperty('createdAt');
    });

    it('повинен повернути 400, якщо дані невалідні (неправильний type)', async () => {
      const res = await request(app)
        .post('/api/planets')
        .send({ ...validPlanet, type: 'InvalidType' })
        .expect(400);

      expect(res.body.message).toBe('Помилка валідації даних');
    });

    it('повинен повернути 400, якщо маса менша нуля', async () => {
      await request(app)
        .post('/api/planets')
        .send({ ...validPlanet, massEarth: -10 })
        .expect(400);
    });
  });
});
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.3	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

describe('GET /api/planets', () => {
  it('повинен повернути порожній масив, якщо база чиста', async () => {
    const res = await request(app).get('/api/planets').expect(200);
    expect(res.body).toEqual([]);
  });

  it('повинен повернути всі створені планети', async () => {
    await request(app).post('/api/planets').send(validPlanet);
    await request(app).post('/api/planets').send(heavyPlanet);

    const res = await request(app).get('/api/planets').expect(200);
    expect(res.body).toHaveLength(2);
  });

  it('повинен фільтрувати планети за типом (query parameter: type)', async () => {
    await request(app).post('/api/planets').send(validPlanet);
    await request(app).post('/api/planets').send(heavyPlanet);

    const res = await request(app).get('/api/planets?type=Gas Giant').expect(200);
    expect(res.body).toHaveLength(1);
    expect(res.body[0].name).toBe('Jupiter');
  });

  it('повинен фільтрувати планети за мінімальною масою (query parameter: minMassEarth)', async ()
=> {
    await request(app).post('/api/planets').send(validPlanet);
    await request(app).post('/api/planets').send(heavyPlanet);

    const res = await request(app).get('/api/planets?minMassEarth=100').expect(200);
    expect(res.body).toHaveLength(1);
    expect(res.body[0].name).toBe('Jupiter');
  });
});

describe('GET /api/planets/heavy', () => {
  it('повинен повернути лише планети з масою >= 10', async () => {
    await request(app).post('/api/planets').send(validPlanet);
    await request(app).post('/api/planets').send(heavyPlanet);

    const res = await request(app).get('/api/planets/heavy').expect(200);
    expect(res.body).toHaveLength(1);
    expect(res.body[0].name).toBe('Jupiter');
  });
});

describe('GET /api/planets/:id', () => {
  it('повинен повернути планету за правильним ID', async () => {
    const createRes = await request(app).post('/api/planets').send(validPlanet);
    const planetId = createRes.body.id;

    const res = await request(app).get(`/api/planets/${planetId}`).expect(200);
    expect(res.body.id).toBe(planetId);
  });

  it('повинен повернути 404, якщо ID не існує', async () => {
    await request(app).get('/api/planets/fake-id-123').expect(404);
  });
});

describe('PATCH /api/planets/:id', () => {
  it('повинен успішно оновити лише передані поля', async () => {
    const createRes = await request(app).post('/api/planets').send(validPlanet);
    const planetId = createRes.body.id;

    const res = await request(app)
      .patch(`/api/planets/${planetId}`)
      .send({ name: 'Earth 2.0' })
      .expect(200);

    expect(res.body.name).toBe('Earth 2.0');
    expect(res.body.type).toBe('Terrestrial');
  });

  it('повинен повернути 404 при спробі оновити неіснуючу планету', async () => {
    await request(app).patch('/api/planets/fake-id').send({ name: 'Nova' }).expect(404);
  });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.3	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		


```
describe('DELETE /api/planets/:id', () => {
  it('повинен видалити планету і повернути статус 204 No Content', async () => {
    const createRes = await request(app).post('/api/planets').send(validPlanet);
    const planetId = createRes.body.id;

    await request(app).delete(`/api/planets/${planetId}`).expect(204);

    await request(app).get(`/api/planets/${planetId}`).expect(404);
  });

  it('повинен повернути 404 при спробі видалити неіснуючу планету', async () => {
    await request(app).delete('/api/planets/fake-id').expect(404);
  });
});
});
```

3.7. Завдання 7: Фільтрація через query-параметри

Розширте додаток підтримкою фільтрації та напишіть тести.

Фільтрацію слід реалізувати на рівні сховища, а не в обробнику маршруту: функція отримання всіх записів приймає опціональні параметри та повертає вже відфільтрований результат. Це правильний поділ відповідальності — у наступних темах, коли сховище замінимо на базу даних, фільтрація природно перетвориться на умови запиту, не змінюючи код маршрутів.

Вимоги:

- Реалізуйте щонайменше 2 query-параметри для фільтрації/пошуку, що мають сенс для вашої предметної області. Назви параметрів обирайте самостійно.
- Параметри можна комбінувати.
- Напишіть тести для кожного параметра та їх комбінацій.
- Додайте один додатковий маршрут з власним шляхом, специфічний для вашої предметної області. Наприклад: GET /planets/habitable, GET /books/bestsellers, GET /recipes/quick. Покрийте його тестами.

```
PASS tests/planet.test.ts
Planet REST API Integration Tests
POST /api/planets
  ✓ повинен створити нову планету і повернути статус 201 (69 ms)
  ✓ повинен повернути 400, якщо дані невалидні (неправильний type) (7 ms)
  ✓ повинен повернути 400, якщо маса менша нуля (5 ms)
GET /api/planets
  ✓ повинен повернути порожній масив, якщо база чиста (8 ms)
  ✓ повинен повернути всі створені планети (19 ms)
  ✓ повинен фільтрувати планети за типом (query parameter: type) (15 ms)
  ✓ повинен фільтрувати планети за мінімальною масою (query parameter: minMassEarth) (14 ms)
GET /api/planets/heavy
  ✓ повинен повернути лише планети з масою >= 10 (18 ms)
GET /api/planets/:id
  ✓ повинен повернути планету за правильним ID (8 ms)
  ✓ повинен повернути 404, якщо ID не існує (5 ms)
PATCH /api/planets/:id
  ✓ повинен успішно оновити лише передані поля (10 ms)
  ✓ повинен повернути 404 при спробі оновити неіснуючу планету (6 ms)
DELETE /api/planets/:id
  ✓ повинен видалити планету і повернути статус 204 No Content (10 ms)
  ✓ повинен повернути 404 при спробі видалити неіснуючу планету (6 ms)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	97.64	93.75	100	97.53	
src	100	100	100	100	
src/app.ts	100	100	100	100	
src/middleware	85.71	50	100	83.33	
src/middleware/error.middleware.ts	71.42	50	100	66.66	12-13
src/middleware/validate.middleware.ts	100	100	100	100	
src/routes	100	100	100	100	
src/routes/planet.routes.ts	100	100	100	100	
src/schemas	100	100	100	100	
src/schemas/planet.schema.ts	100	100	100	100	
src/storage	100	100	100	100	
src/storage/planet.storage.ts	100	100	100	100	

```
Test Suites: 1 passed, 1 total
Tests: 14 passed, 14 total
Snapshots: 0 total
Time: 4.127 s, estimated 5 s
Ran all test suites.
```